

Draw Tool

08



We use a variety of tools to create digital drawings. We can use code to create a drawing tool. Let's learn how!



We have learnt to use different computer applications to draw. It's easy and saves us a lot of paper. One such application is Paint, but we can make a drawing application ourselves too!

A computer application or software is a set of instructions written in a programming language to perform a specific task or an operation on a computer.

A computer contains different types of applications. Some applications run invisibly in the background while others need to be opened when needed.

For example: MS Word, Excel, Paint, etc.



Paint



Windows Media Player



Notepad



Word



Excel



PowerPoint

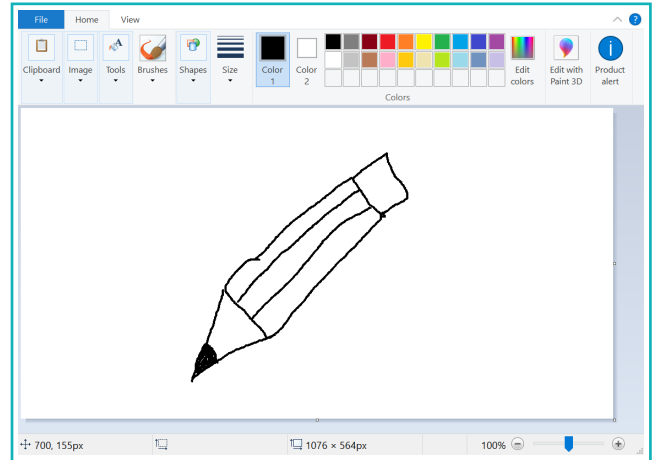


Photos

8. Draw Tool

Computer applications are created using code in various programming languages. JavaScript is one such language used to create a web-based application.

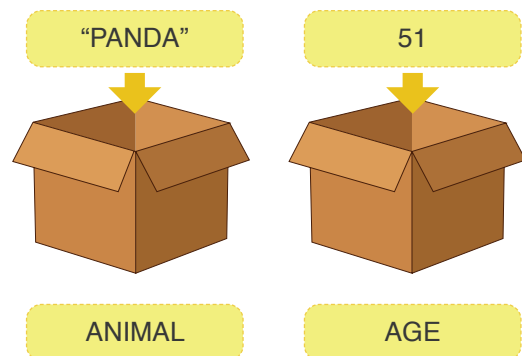
In the last lesson, we learnt to draw different shapes using code. Today, we will create a drawing application where we can use a mouse to draw.



The part of our screen where we can draw will be a canvas. We will use a mouse to draw. The cursor's position will be determined by x and y coordinates on the canvas. These coordinates can be saved using variables.

Variables

Variables are entities that can store values. Characters, like 'a', 'b', or words can be used to name variables. A value is stored in the variable and we can use a name to represent it.



For example: A variable called 'age' stores age in numbers. Whereas, a variable 'name' can store your name. This means that variables can save values as well as words.

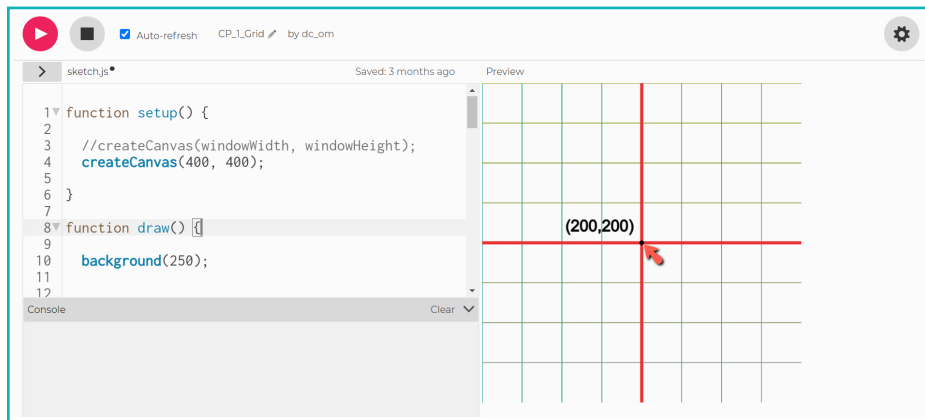
A few default variables are provided in the programming language that store specific data based on system data or user input.

Default variables

Default variables store values of some parameters provided by the computer. p5js has a few variables which appear in pink colour in the editor.

For example, **mouseX** and **mouseY** are default variables in p5js. They store the value of the current x and y positions of the mouse cursor on the canvas.

When a cursor is at position 200,200 on the 400x400 canvas, the value inside mouseX and mouseY will be 200 and 200 respectively.



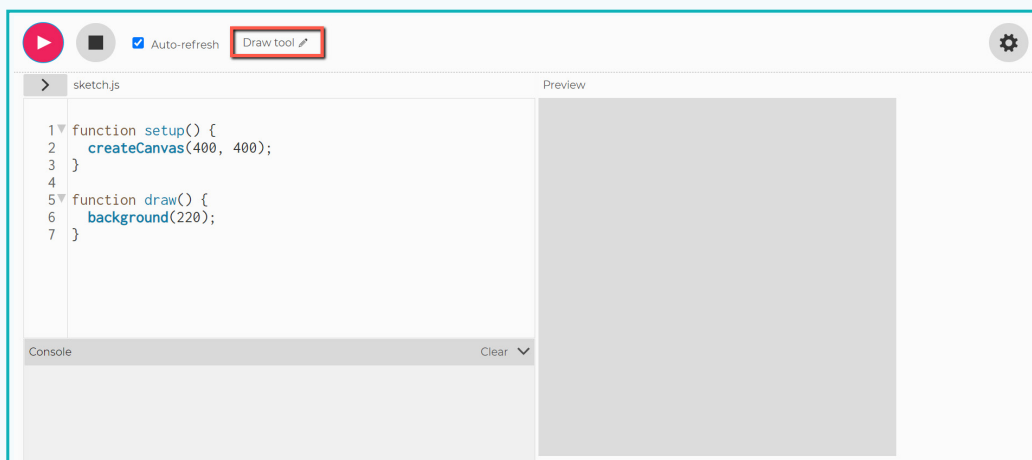
Now we know that a point can be drawn at the mouse location using default variables mouseX and mouseY. Let's create a drawing application using p5js.

Activity

Create a drawing application using p5.js

1 Complete initial setup

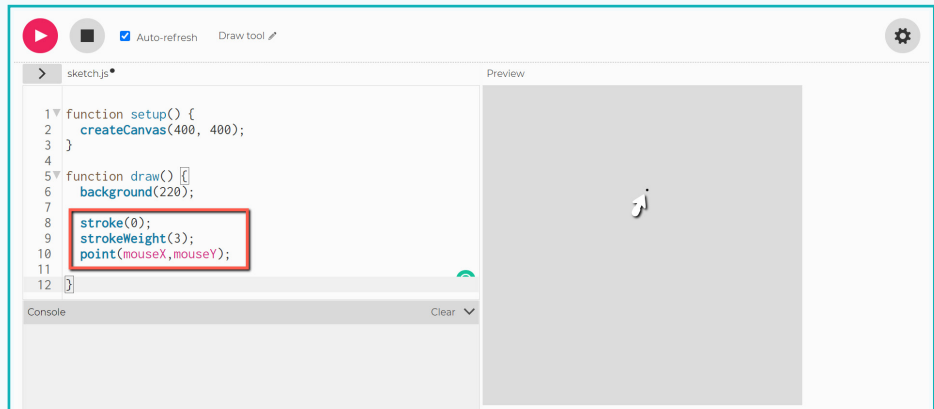
- Open the p5js editor and name the project 'Draw tool'.



Activity

2 Draw a point with the mouse

- Add `strokeWeight(3)` and `stroke(0)` to set line thickness and point colour as black.
- Add `point(mouseX, mouseY)` to draw a point at the cursor location.



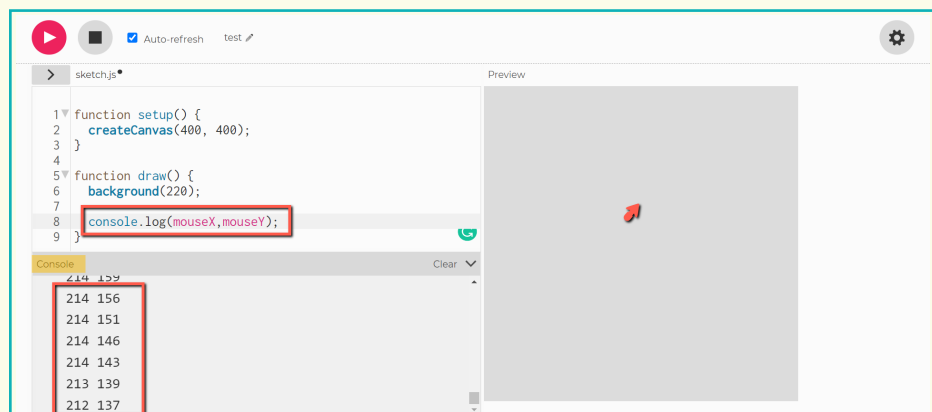
We can see a point that moves with the cursor. The location values of this are `mouseX` and `mouseY`.

Note:

In the last lesson, we learned about the console area which help us debug the code. `console.log()` is used to display the value inside the variable.

console.log() – This command prints the input inside `log(...)` in the console area.

The code `console.log(mouseX, mouseY)` displays the x and y position of the cursor in the console. We use this to find x and y positions while drawing shapes and the values of points on the canvas.



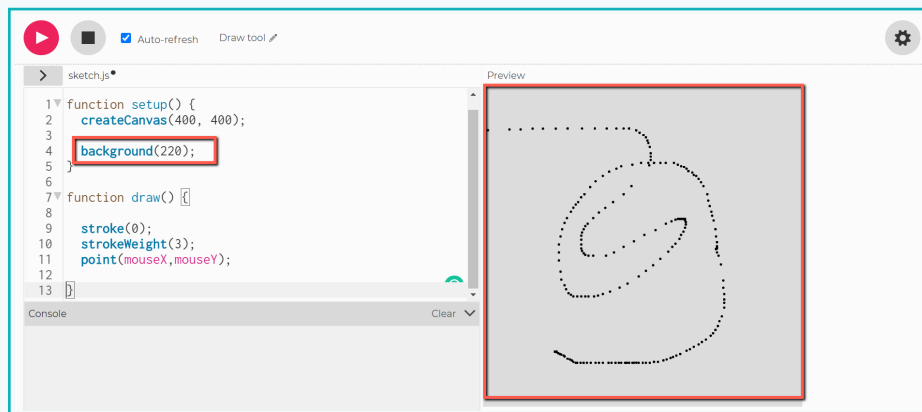
We've seen that the point at the cursor location moves with the cursor. But points drawn at earlier locations vanish. This is because of the `background(220)` command. Code inside `draw()` function runs in a loop. If a new point is drawn, the `background()` command applies the background colour over the earlier point, causing it to disappear.

To avoid this, we use `background()` command in `setup()` function. This way it runs only once at the start of the code.

Activity

3 Draw a line using points

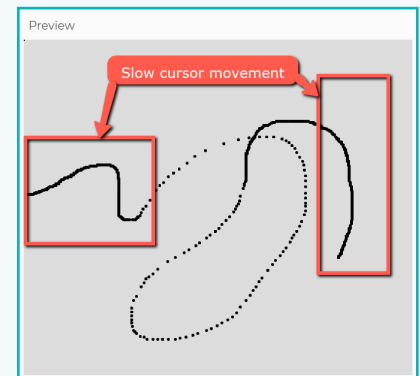
- Cut the background command from draw() and paste in setup().



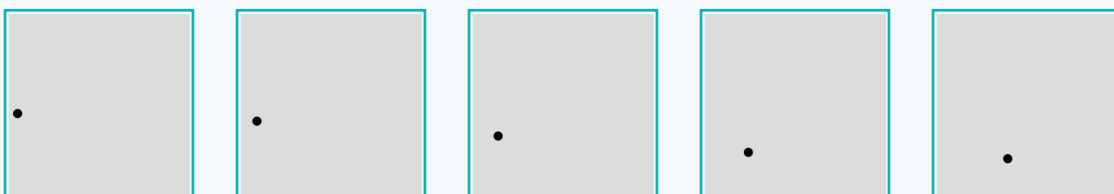
Now, we can see points drawn along the path of cursor movement, but we don't have a continuous line.

This is because the speed of showing points is slower than the speed of cursor movement. Hence, we cannot see a point drawn continuously along the path. If we move the cursor slowly, we will see continuous points drawn.

This happens because the speed of displaying points is sometimes slower than the cursor movement. If we move the cursor slowly, we can see continuous points as a line.



The code we write, repeatedly and continuously renders the visual output after we click run. A single visual output rendered at a time is called a **frame**.

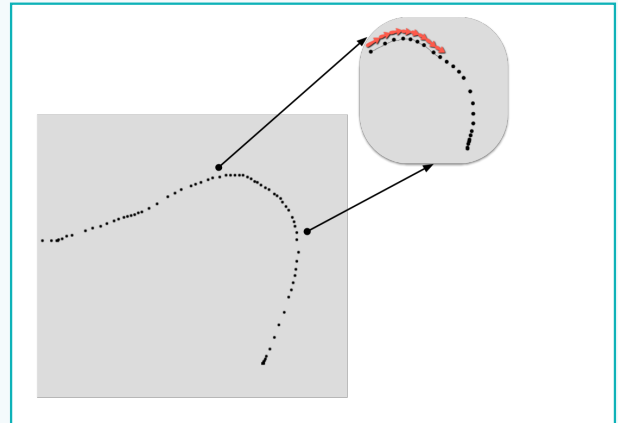


These images show frames where the point is moving. If they are displayed quickly, one after the other, we can view the continuous movement of the point.

The process that shows visual output in the form of an image or animation is called **rendering**.

Activity

To draw a continuous line along with the cursor movement, we draw lines between two points in two consecutive frames.

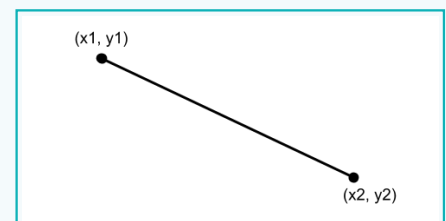


To get x and y values of the point drawn earlier, we will use pmouseX and pmouseY variables. These are default variables which store the x and y position values of the mouse cursor in the previous frame.

We can draw a line between a point in the current frame (position mouseX and mouseY) and a point in the previous frame (position pmouseX and pmouseY).

We draw a line with the line command. Two vertices are used as an input.

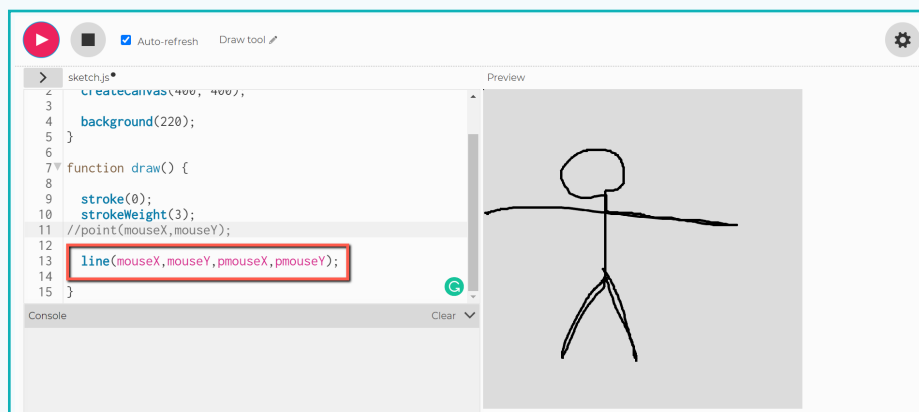
line (x1,y1,x2,y2);



To set the line properties, use stroke() with color as an input of a single value between 0 to 255 for grey scale colour. Use strokeWeight() with a thickness value in pixels to change the thickness of the line.

4 Draw a continuous line using the mouse pointer

- Comment out the command to draw point(mouseX, pmouseY);
- Add command line(mouseX, pmouseY, pmouseX, pmouseY);



This way, we can draw on the canvas just like pen tool in paint! Use your imagination and make some drawings with this.

Exercise

Tick mark ☒ the correct answer

- 1 A single visual output shown at a specific instant is called a _____ .
- | | | | |
|------------|--------------------------|----------------|--------------------------|
| a. Frame | ☐ | b. Rendering | ☐ |
| c. Drawing | ☐ | d. Application | ☐ |
- 2 A point drawn at the cursor location moves with the cursor but doesn't leave a trail because the background() function is in _____ function.
- | | | | |
|-----------------|--------------------------|-----------------|--------------------------|
| a. setup | ☐ | b. draw | ☐ |
| c. mouseDragged | ☐ | d. None of them | ☐ |
- 3 _____ default variables store the x and y position of the mouse cursor from the previous frame.
- | | | | |
|-------------------|--------------------------|---------------------|--------------------------|
| a. mouseX, mouseY | ☐ | b. pmouseX, pmouseY | ☐ |
| c. both | ☐ | d. None of them | ☐ |
- 4 _____ default variables store the x and y position of the mouse cursor from the current frame.
- | | | | |
|-------------------|--------------------------|---------------------|--------------------------|
| a. mouseX, mouseY | ☐ | b. pmouseX, pmouseY | ☐ |
| c. both | ☐ | d. None of them | ☐ |

Activity

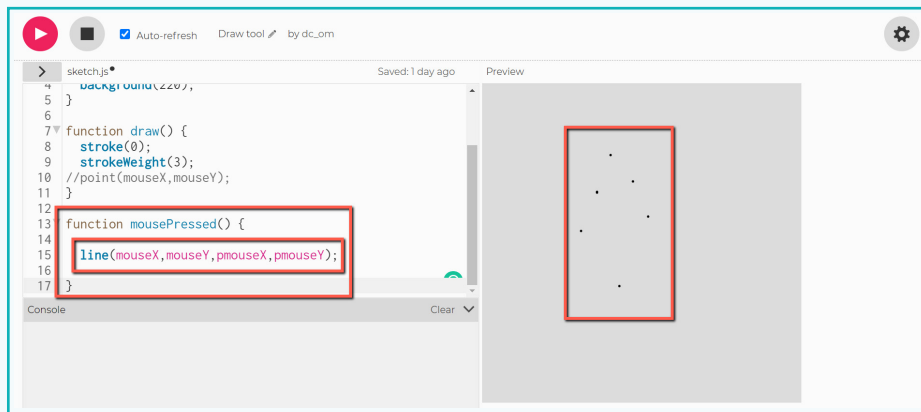
In the method we just learnt, the cursor draws continuously. We cannot control where to start drawing and where to stop. Let's learn to change it so that when the mouse button is pressed down, it will draw. When we let go of the button, it will stop drawing.

mousePressed() is a default function that is called when we click the mouse button.

Activity

5 Draw a continuous line when the mouse button is pressed

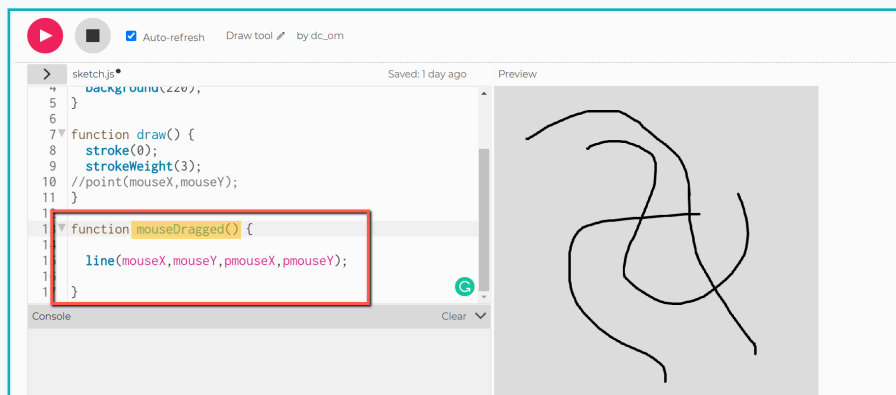
- Add mousePressed() function just like setup() or draw().
- Inside it, add code to draw a line.



The result is not as desired. The function only works when the mouse button is clicked and draws only a dot. Because it doesn't remain active for as long as the button is pressed. Unless we modify it, it will not give us our desired result.

Instead, we will use another default function called mouseDragged(). The function remains active as long as the mouse button is pressed.

- Replace mousePressed() with mouseDragged() in the code.



Now, we can use our mouse to draw just like we do in other drawing applications! Our first software 'drawTool' is ready.

Drawing like this is a lot of fun. But right now, we can't erase what we have drawn. We have to stop the program and run it again to make a new drawing.

To overcome this, we will use keyPressed() default function to use a key as an eraser. This runs the code written inside only once when any key is pressed.

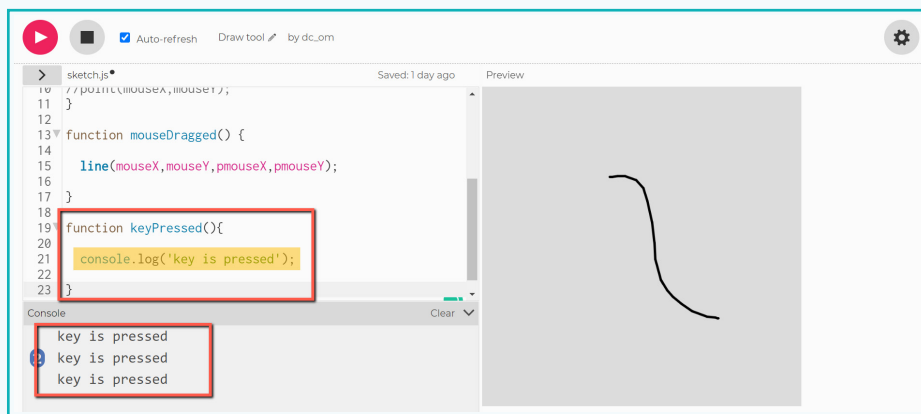
Activity

6 Add eraser function

- Add keyPressed() function.
- Add `console.log('key is pressed');` inside the function.

Note:

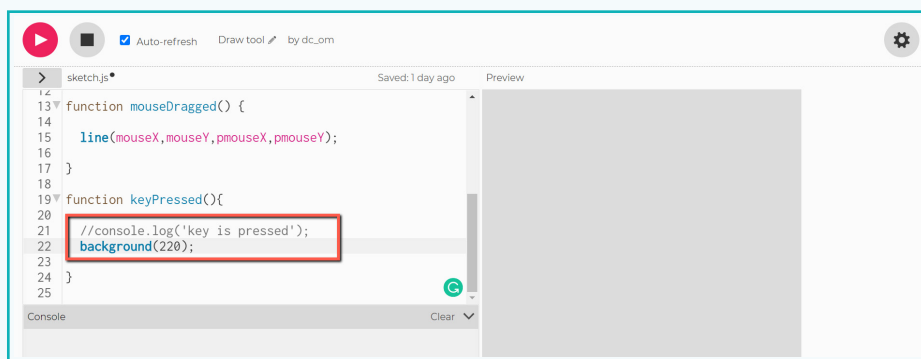
Text is always added in single inverted commas ' '. Once added, the text appears in green.



Draw something and then press any key on keyboard. Every time we press any key, the text we added in `console.log()` will be displayed.

Now that we know how `keyPressed()` works, we will add code into it.

- Comment out `console.log()` command.
- Add `background(220);` to apply background colour to clear the drawing.



We have created a draw tool with an erase option. Try out different pen thicknesses and colours in `strokeWeight()` and `stroke()` respectively.

Our draw tool is now complete! Experiment with drawings of your own.

Exercise

State whether True or False

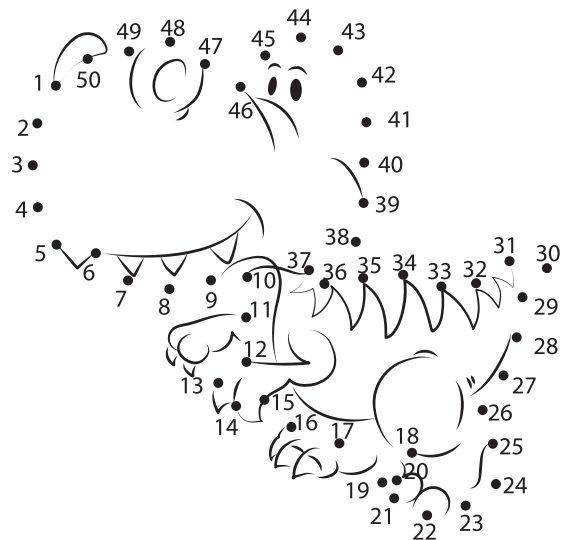
5 mouseDragged() function is invoked when a mouse is dragged with the button pressed.

▲ _____

6 To draw a continuous line along with the mouse cursor, point() function is used.

▲ _____

We have created a draw tool. We will modify it to play a fun “Connect the Dots” game. Let’s look for a copyright free image online and upload it.



To modify our tool, we will upload an image and use it in the code to display on Canvas using a variable. We have seen default variables store data. We can also create a variable where we can store a value we want. Such variables are called User defined variables.

Variable so far are used to store data in the form of numbers or words. We can also declare the variable at the beginning of the code before setup() function to store an image. A variable is declared using a keyword ‘var’ and the variable name.

For example:

```
var name; var x;
```

We will assign an initial value to the variable inside setup() function.

```
x=10;
```

```
name='Raghav';
```

Activity

7 Add an image file

We will create a variable called img using the keyword var to store the image we upload.

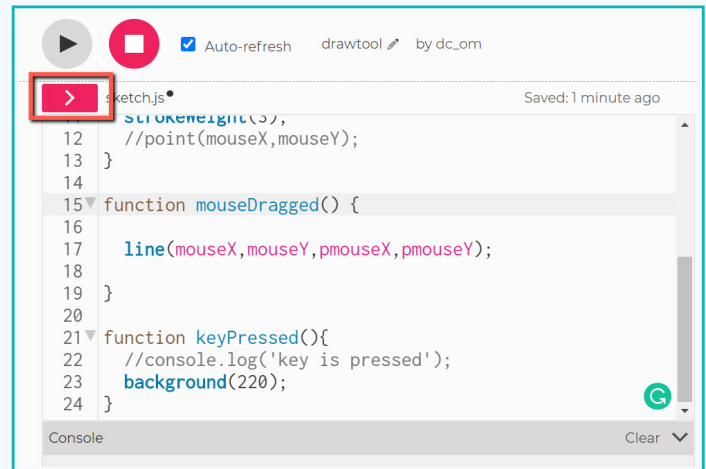
```

1 var img;
2
3 function setup() {
4   createCanvas(400, 400);
5
6   background(220);
7 }
8
9 function draw() {
10  stroke(0);
11  strokeWeight(3);
12  //point(mouseX,mouseY);
13

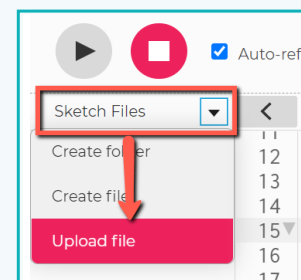
```

To upload the image file in the code, we have to add it in sketch files first.

- Click on the arrow at the top left of the editor window.

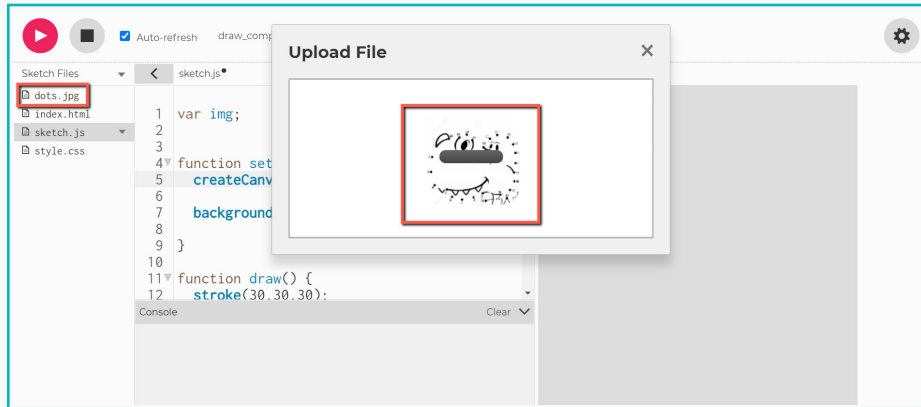


- Click on 'Sketch Files' and select upload files.



Activity

- c. Select the image from the computer. After it uploads, the image will appear in sketch files.



We can use a default function `preload()` and add code to store the picture in 'img' variable.

Note:

`Preload()` function is used to load external files before the code is executed. It is always added directly before `setup()` function.

```
function preload() {  
  // preload() runs once  
}
```

If a `preload` function is defined, `setup()` will wait until the load command inside the function is complete. Only load commands like `loadImage`, `loadFont`, `loadStrings`, etc. will work inside the `preload` function.

`loadImage('image_file_name')` – When we assign this command to a variable, it stores the image from sketch files inside a variable.

For example: To add an image inside 'img' variable, we can use the command

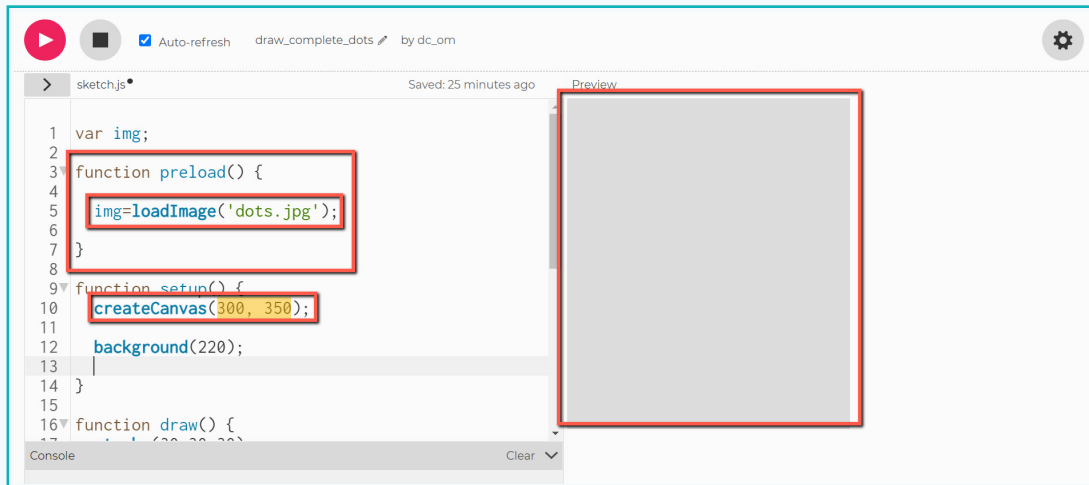
```
img=loadImage('dots.jpg');
```

8 Display image using variable

- Add function `preload()` before `setup()` in the code.
- Use `loadImage()` to load the image from sketch files and save inside 'img' variable.
- Since the image is in sketch files, we can directly write the file name as input to the `loadImage()` command in single inverted commas.

Activity

Change the canvas size to match the image dimensions. In this case it will be (300,350)

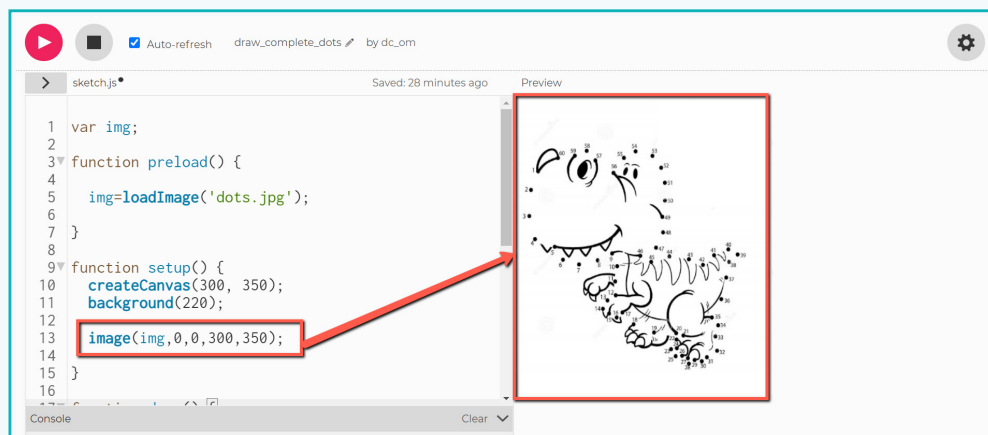


To display the image on the canvas once at the beginning, add `image(imageVariable,x,y,width,height);` command in `setup()` function.

The first parameter is the variable in which the image is stored.

X and y refer to the starting point of the image's top left corner.

The next two parameters set the width and height of the image. If the last two parameters are not mentioned, the image will have its original dimensions.

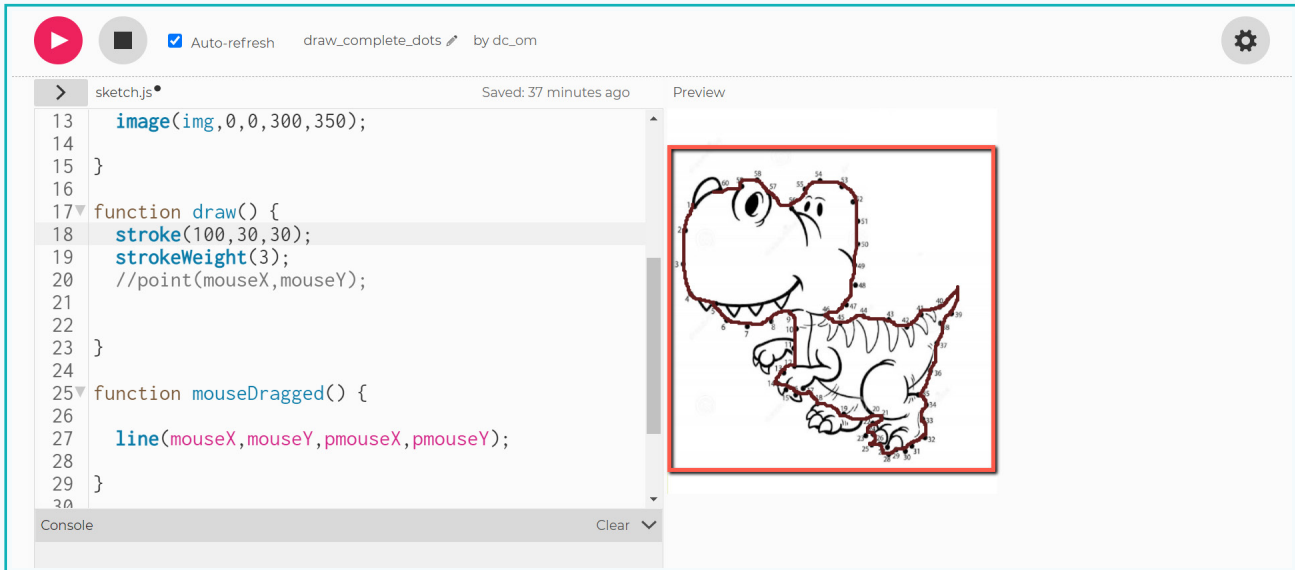


Note:

We have added `image()` command in `setup()` so that is called only once. It will display whatever we draw on the image to complete the dots.

Activity

Now, we can use the draw tool to join the dots and complete the drawing. Share dotted images with your friends and ask them to complete them in p5js.



In this lesson, we learnt about variables. We used default variables to draw on the canvas with a mouse. Then, we modified the tool by creating our own variables to make a “Connect the Dots” game.

Exercise

Answer in one line

- 7 Write code for a function which adds commands to load a file before the code runs. It should be added before setup() function.



- 8 Write a command to use in preload function to upload image 'drawing.png' stored in sketch files inside a variable called 'image1'.



- 9 Write a command to display the image stored inside 'image1' variable at x=100, and y=100 location, with width=400 height=400.



User interaction is the key element of any software and such fun computer tools can be created with coding.



Tech Flash

- **Variable** : An entity or character that stores data in the form of numbers or words. It is defined using the key word 'var'.
- **Console log** : A log created using `console.log()` command to check variable values and find bugs in the code.
- **Rendering** : A process of showing visual output in the form of an image or animation created using an application.
- **Frame** : A single visual output rendered at a specific time.
- **Frame Rate** : The number of frames displayed in one second. Most movies are filmed at 24 frames per second.
- **Preload** : A function is used to load external files like images, string etc. before the code runs. It is always added before the `setup()` function. It helps smoothly execute the code since the required files are preloaded in the code.